Erlind Sejdiu
106357

SYCL

Portable heterogene Programmierung mit C++

Problemstellung, Anwendungsfälle und Programmiermodell

Seminararbeit
der Hochschule Karlsruhe
im Master Studiengang Informatik
im Wintersemester 2026/27

Betreuer: Prof. Dr.-Ing. Holger Vogelsang
Abgabedatum: 30.11.2026

Inhaltsverzeichnis

1 Problemstellung **1**

2 Forschungsfrage **1**

3 Stand der Technik **2**

4 Methode **3**

5 Konzepte des Programmiermodells **4**
5.1 Host, Gerät und Kernel 4
5.2 Speichermodell 5
5.3 Asynchrone Ausführung und Abhängigkeiten 6
5.4 Hierarchische Ausführung und lokaler Speicher 7
5.5 Abbildung auf die Hardware 8
5.6 Geräteauswahl, Backends und JIT-Kompilierung 9
5.7 Fehlerbehandlung 10
5.8 Zusammenfassung 10

6 Anwendung auf die Matrixmultiplikation **11**
6.1 Stufe 0 – Ausgangsprogramm 11
6.2 Stufe 1 – Der erste Kernel 11
6.3 Stufe 2 – Expliziter Gerätespeicher 12
6.4 Stufe 3 – Kachelung im lokalen Speicher 13
6.5 Stufe 4 – Gerätewechsel und Kachelgröße 15

7 Ergebnisse **16**

8 Diskussion **17**

9 Schlussfolgerung **18**

Literaturverzeichnis **19**

A Quellcode der Matrixmultiplikation **21**
A.1 Stufe 0 – Sequentielles Ausgangsprogramm 21
A.2 Stufe 1 – USM shared und `parallel_for` 22
A.3 Stufe 2 – USM device und explizites Kopieren 23
A.4 Stufe 3 – Kachelung im lokalen Speicher 24
A.5 Gemeinsame Hilfsfunktionen (`common.hpp`) 26

Abbildungsverzeichnis

1 SYCL-Codebasis für alle Backends 1
2 Grundaufbau (Stufe 1). Der Host reicht einen Kernel über eine Queue ein, das Gerät führt ihn als viele Work-Items aus 4
3 Speichermodell, aufbauend auf Stufe 1. Neu sind Host- und Gerätespeicher mit den Matrizen A, B, C. Bei USM device kopiert der Host die Daten ausdrücklich (durchgezogen), bei USM shared verschiebt die Laufzeit sie automatisch (gestrichelt). Stufe 1 verwendet USM shared 5
4 Speicherebenen in SYCL, aufbauend auf Abbildung 3. Neu sind die Ebenen auf dem Gerät. Nach unten werden sie kleiner, schneller und von weniger Work-Items geteilt 6
5 Getrennter Speicher und Abhängigkeiten (Stufe 2). Neu sind der Gerätespeicher und explizite Kopien, deren Abhängigkeiten als Pfeile in der Queue stehen. Kopie A und Kopie B hängen nicht voneinander ab und können gleichzeitig laufen 7
6 Eine nd_range mit globalem Bereich 8×8 und lokalem Bereich 4×4. Dick umrandet sind die vier Work-Groups 7
7 Hierarchische Ausführung (Stufe 3). Neu sind die Command Group sowie Work-Groups mit eigenem lokalem Speicher und Barriere 8
8 Abbildung der SYCL-Ausführungshierarchie auf GPU und CPU 9
9 Übersetzung und Ausführung. Der Gerätecode liegt als Zwischendarstellung vor und wird erst beim ersten Start für das gewählte Gerät übersetzt 9
10 Synchrone und asynchrone Fehler. Der synchrone Fehler wird beim Einreichen gemeldet, der asynchrone erst, wenn der Host auf das Gerät wartet 10
11 Zuordnung der Work-Items zu den Elementen von C in Stufe 1 13
12 Kachelung in Stufe 3. Eine Work-Group durchläuft die Kachelzeile von A und die Kachelspalte von B in n/T Schritten 14
13 Effizienz und Performance-Portabilität je Kachelgröße bei $n = 2048$ . . 17

Tabellenverzeichnis

1 SYCL im Vergleich zu anderen Programmiermodellen ((✓) = eingeschränkt) 2
2 Stufen der Portierung und die dabei eingeführten Konzepte 3
3 Varianten von Unified Shared Memory [3] 5
4 Konzepte von SYCL, die von ihnen adressierten Probleme und die Stufe, in der sie bei der Portierung eingeführt werden 11
5 Änderungen in Stufe 1 gegenüber Stufe 0 12

6 Leistung je Stufe in GFLOPS (Median aus fünf Läufen, Stufe 0 auf einem Kern der jeweiligen Host-CPU) 16
7 Laufzeit je Stufe in ms (Median aus fünf Läufen). Ab Stufe 2 sind Kernel- und Transferzeit getrennt erfasst 16

1 Problemstellung

Rechenleistung liegt heutzutage in verschiedenen Formen vor. Neben der CPU stehen in fast jedem Rechner GPUs und andere technische Komponenten zur Verfügung, die datenparallele Aufgaben schnell bearbeiten können. Die Software, um diese Geräte jeweils anzusprechen, kann variieren, denn jeder Hersteller bringt sein eigenes Programmiermodell mit. Für die Software ergeben sich daraus drei Probleme.

1. **Herstellerbindung.** CUDA läuft nur auf NVIDIA-GPUs [1]. AMD (HIP) und Intel (Level Zero) haben eigene Schnittstellen.
2. **Mehrfache Codebasen.** Wer mehrere Hersteller unterstützen will, pflegt mehrere Implementierungen parallel (Abbildung 1a).
3. **Abstraktion gegen Kontrolle.** Offene Alternativen sind entweder sehr hardwarenah und aufwendig wie OpenCL [2] oder lassen wie OpenMP-Direktiven wenig Kontrolle über Speicher und Ausführung.

2 Forschungsfrage

SYCL ist ein offener Standard der Khronos Group, der diese Probleme lösen will. Mit einem einzigen Quellcode in Standard-C++ soll auf CPUs und GPUs verschiedener Hersteller dasselbe Programm laufen [3].

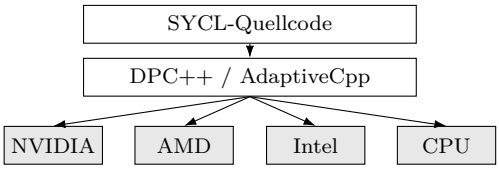


Abbildung 1: SYCL-Codebasis für alle Backends

Ob ein Programm überall läuft, ist jedoch etwas anderes, als ob es überall schnell läuft. Diese Arbeit untersucht deshalb die folgende Frage.

> Wie adressiert das Programmiermodell von SYCL die drei genannten Probleme, und erreicht ein schrittweise nach SYCL portiertes Programm neben funktionaler Portabilität auch Performance-Portabilität zwischen CPU und GPU?

Daraus ergeben sich zwei Teilfragen. (F1) Welche Konzepte des Programmiermodells werden bei einer Portierung benötigt, und welches der drei Probleme lösen sie? (F2) Welche Stufen einer Portierung lassen sich unverändert auf verschiedenen Geräten ausführen, und welche Parameter bleiben gerätespezifisch?

3 Stand der Technik

SYCL basiert auf C++17 [3]. Konkrete Implementierungen des Standards wie DPC++ [4] und AdaptiveCpp [5], [6] bilden den Standard über Backends wie CUDA, HIP, Level Zero, OpenCL oder OpenMP auf die Hardware ab und übersetzen den Gerätecode meist erst beim ersten Start just in time für das konkrete Gerät [7]. Den drei Problemen setzt SYCL vier Entwurfsziele entgegen. Portabilität (ein Quellcode, viele Backends), Offenheit (ein Standard mit mehreren unabhängigen Implementierungen), Produktivität (Host- und Gerätecode in einer Datei in Standard-C++) und Kontrolle (Hardwarekonzepte wie lokaler Speicher bleiben zugänglich). Tabelle 1 ordnet SYCL entlang dieser Ziele ein. Kokkos [8] und RAJA [9] verfolgen ähnliche Ziele, sind aber kein offener Industriestandard.

	herstellerunabh.	**offener Standard**	**Standard-C++**	**Kontrolle**
CUDA	–	–	(✓)	✓
OpenCL	✓	✓	–	✓
OpenMP-Offload	✓	✓	✓	(✓)
Kokkos/RAJA	✓	–	✓	(✓)
SYCL	✓	✓	✓	✓

Tabelle 1: SYCL im Vergleich zu anderen Programmiermodellen ((✓) = eingeschränkt)

Eingesetzt wird SYCL vor allem im wissenschaftlichen Rechnen auf Systemen verschiedener Hersteller, etwa auf dem Supercomputer Aurora oder in GROMACS, bei der Migration von CUDA-Anwendungen mit dem Werkzeug SYCLomatic und in portablen Bibliotheken wie oneMKL [10]. Frühere Studien fanden SYCL auf vielen Plattformen konkurrenzfähig mit herstellerspezifischem Code, aber deutlich abhängig von Implementierung und Parametrierung [11], [12]. Als Maß für Performance-Portabilität hat sich die Kennzahl von Pennycook, Sewall und Lee etabliert [13]. Offen bleibt in diesen Arbeiten meist, welcher Schritt einer Portierung welchen Effekt auf welchem Gerät hat. Hier setzt die vorliegende Seminararbeit an.

4 Methode

Untersucht wird die Multiplikation $C = A \cdot B$ zweier $n \times n$-Matrizen. Das Problem benötigt $2n^3$ Gleitkommaoperationen bei $3n^2$ Werten Datenvolumen, ist gut parallelisierbar und leicht auf Korrektheit prüfbar. Ein sequentielles C++-Programm wird in Stufen nach SYCL überführt, wobei jede Stufe nur so viele neue Konzepte einführt, wie für ihren Schritt nötig sind (Tabelle 2). Abschnitt 5 erklärt diese Konzepte zunächst allgemein und in der Reihenfolge, in der sie gebraucht werden. Abschnitt 6 zeigt anschließend, wie sie in den einzelnen Stufen umgesetzt sind. Das beantwortet F1. Zugleich lässt sich die Wirkung jedes Schritts getrennt messen, was F2 beantwortet.

Stufe	**Änderung**	**Neue Konzepte**
0	Sequentielles Ausgangsprogramm auf einem CPU-Kern	–
1	Ein Work-Item je Element von C	Host und Gerät, Queue, Kernel, Indexraum, USM shared
2	Getrennter Gerätespeicher, explizites Kopieren	USM device, explizite Kopien, Events, Abhängigkeiten, Buffer
3	Kachelung im lokalen Speicher	Command Group, Work-Group, lokaler Speicher, Barriere
4	Kein neuer Code, nur Gerätewechsel und Variation der Kachelgröße	Geräteauswahl, Backends, Geräteabfragen, Fehlerbehandlung

Tabelle 2: Stufen der Portierung und die dabei eingeführten Konzepte

Gemessen wird mit DPC++ [4] auf drei Geräten. Zumal auf einem AMD Ryzen 5 5600X (sechs Kerne, OpenCL-Backend), einer NVIDIA GeForce RTX 5060 Ti und einer NVIDIA L40S auf dem DACHS-Cluster der Hochschule Esslingen (beide CUDA-Backend). Jede Stufe läuft für $n \in \{256, 512, 1024, 2048\}$. Nach einem getrennt erfassten ersten Lauf, der die JIT-Kompilierung enthält, folgen fünf Messläufe, von denen der Median berichtet wird. Kernel- und Transferzeit werden getrennt erfasst.

5 Konzepte des Programmiermodells

Dieses Kapitel erklärt die Konzepte von SYCL unabhängig von der Matrixmultiplikation. Die Reihenfolge folgt den Stufen aus Tabelle 2, sodass jedes Konzept dort steht, wo es bei der Portierung zum ersten Mal gebraucht wird. Die Abbildungen 2, 3, 5 und 7 bauen aufeinander auf. Bereits Bekanntes ist grau gezeichnet, Neues dick schwarz. Wie die Konzepte im Quellcode der Matrixmultiplikation umgesetzt sind, zeigt Abschnitt 6.

5.1 Host, Gerät und Kernel

SYCL unterscheidet zwei Rollen. Der Host ist die CPU, auf der das Hauptprogramm läuft. Ein Gerät ist die Hardware, die rechnet, etwa eine GPU oder auch die CPU selbst. Ein Kernel ist eine Funktion, die auf dem Gerät vielfach parallel ausgeführt wird. Jede einzelne Ausführung heißt Work-Item. Wie viele Work-Items es gibt und wie sie angeordnet sind, legt der Indexraum fest. Jedes Work-Item kennt seine Position im Indexraum und wählt darüber die Daten aus, die es bearbeitet. Ein Kernel beschreibt also nur, was ein einzelnes Work-Item tut. Der Host übergibt Kernel nicht direkt an das Gerät, sondern über eine Queue. Jede Queue ist mit genau einem Gerät verbunden (Abbildung 2). Das Einreichen ist asynchron. Der Aufruf kehrt sofort zurück, und der Host kann weiterarbeiten, bis er das Ergebnis tatsächlich braucht und ausdrücklich darauf wartet.

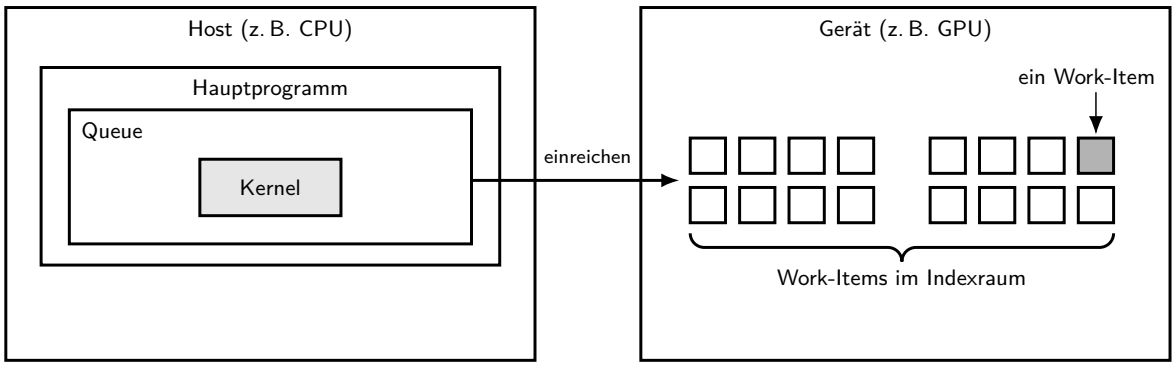


Abbildung 2: Grundaufbau (Stufe 1). Der Host reicht einen Kernel über eine Queue ein, das Gerät führt ihn als viele Work-Items aus

Host- und Gerätecode stehen in derselben C++-Quelldatei (Single-Source) [3]. Der Kernel wird meist als Lambda-Ausdruck geschrieben und kann Variablen des Hauptprogramms als Kopie übernehmen. Der Compiler erkennt den Gerätecode und übersetzt ihn zusätzlich für die Zielgeräte, entweder in einem eigenen Durchlauf je Ziel oder, wie AdaptiveCpp, in einem einzigen gemeinsamen Durchlauf [7]. Weil Geräte keine vollständige C++-Umgebung besitzen, gelten im Kernel Einschränkungen. Ausnahmen, Rekursion, virtuelle Funktionsaufrufe und dynamische Speicheranforderung sind dort

nicht erlaubt. Gegenüber CUDA oder OpenCL entfällt damit eine eigene Kernelsprache. Es bleibt bei einer Codebasis in Standard-C++.

5.2 Speichermodell

Host und Gerät besitzen häufig getrennte Speicher, etwa den Arbeitsspeicher des Rechners und den Grafikspeicher einer GPU. SYCL bietet zwei Wege, Daten zwischen ihnen zu verwalten, Unified Shared Memory (USM) und Buffer.

USM arbeitet mit gewöhnlichen C++-Zeigern, die je nach Variante auf Host, Gerät oder beiden gültig sind (Tabelle 3). Bei USM shared gilt derselbe Zeiger auf Host und Gerät. Die Laufzeit verschiebt die Daten automatisch dorthin, wo zugegriffen wird. Das ist bequem, verbirgt aber, wann und wie viel kopiert wird. Bei USM device liegen die Daten nur im Gerätespeicher, und der Host kopiert sie ausdrücklich hin und zurück. Das verlangt mehr Code, macht den Transfer aber sichtbar und steuerbar (Abbildung 3).

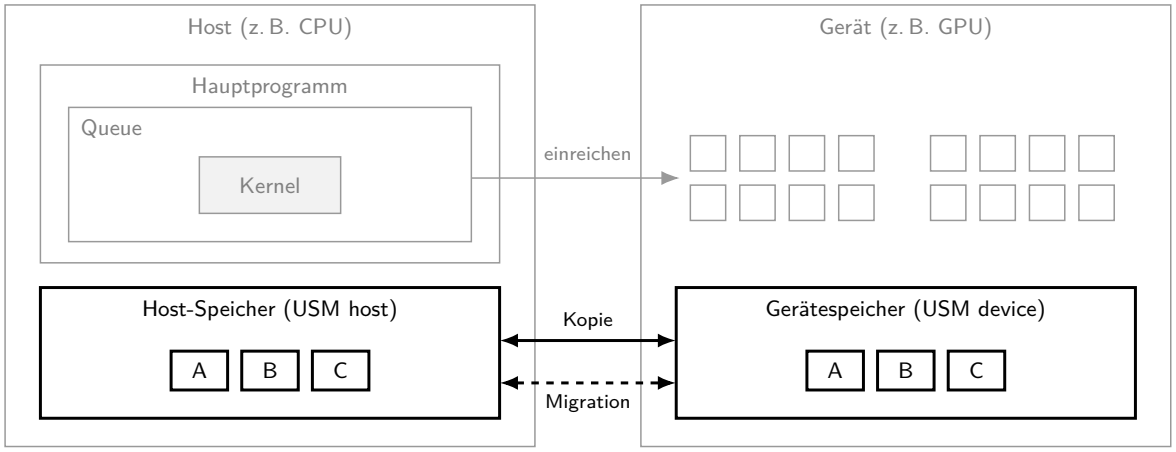


Abbildung 3: Speichermodell, aufbauend auf Stufe 1. Neu sind Host- und Gerätespeicher mit den Matrizen A, B, C. Bei USM device kopiert der Host die Daten ausdrücklich (durchgezogen), bei USM shared verschiebt die Laufzeit sie automatisch (gestrichelt). Stufe 1 verwendet USM shared

Variante	**Speicherort**	**Zugriff**	**Datentransfer**
USM host	Host-Speicher	Host und Gerät, Gerät über den Bus (langsam)	keiner
USM device	Gerätespeicher	nur Gerät	explizit über Kopierbefehle
USM shared	wandert zwischen Host und Gerät	Host und Gerät	automatisch durch die Laufzeit

Tabelle 3: Varianten von Unified Shared Memory [3]

Buffer sind die ältere, abstraktere Alternative. Ein Buffer kapselt die Daten, und ein Kernel greift nur über Accessoren darauf zu, die zugleich angeben, ob gelesen, geschrie-

ben oder beides wird. Aus diesen Angaben leitet die Laufzeit selbst ab, wann Daten kopiert werden müssen und welche Kernel voneinander abhängen. Der Programmierer verliert dafür die direkte Kontrolle über Zeitpunkt und Umfang der Kopien [10].

Auf dem Gerät selbst gibt es mehrere Speicherebenen (Abbildung 4). Der globale Speicher ist groß, für alle Work-Items sichtbar, aber vergleichsweise langsam. Der lokale Speicher ist klein und schnell und wird nur von den Work-Items einer Work-Group geteilt (Abschnitt 5.4). Der private Speicher, auf der GPU meist Register, gehört einem einzelnen Work-Item. Je weiter unten eine Ebene liegt, desto schneller und kleiner ist sie. Daten so zu verteilen, dass häufig gelesene Werte in den schnellen Ebenen liegen, ist der wichtigste Hebel für Leistung [14].

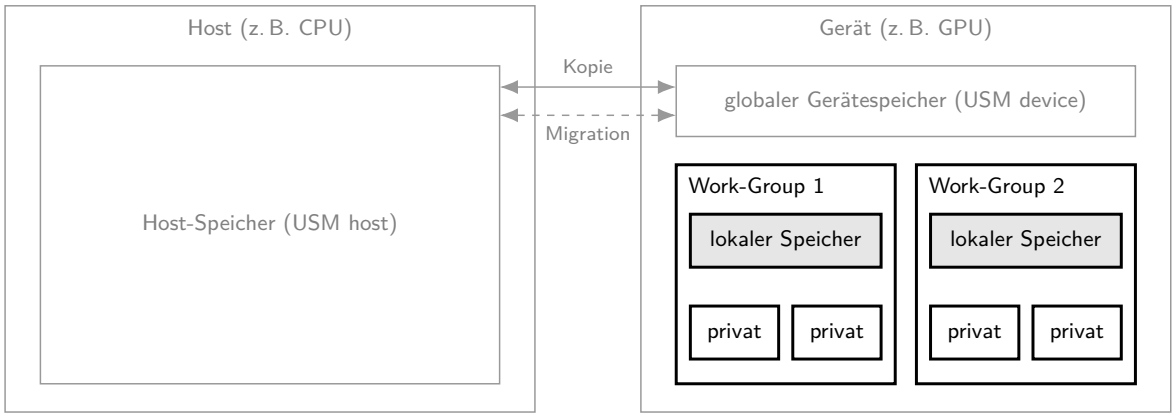


Abbildung 4: Speicherebenen in SYCL, aufbauend auf Abbildung 3. Neu sind die Ebenen auf dem Gerät. Nach unten werden sie kleiner, schneller und von weniger Work-Items geteilt

5.3 Asynchrone Ausführung und Abhängigkeiten

Da Kopien und Kernel asynchron eingereicht werden, muss die Laufzeit wissen, in welcher Reihenfolge sie ausgeführt werden dürfen. Ein Kernel darf zum Beispiel erst starten, wenn seine Eingabedaten auf dem Gerät angekommen sind. Jeder Auftrag an die Queue liefert dafür ein Event. Spätere Aufträge können angeben, auf welche Events sie warten. Aus diesen Abhängigkeiten baut die Laufzeit einen gerichteten azyklischen Graphen und führt Aufträge, die nicht voneinander abhängen, gleichzeitig aus (Abbildung 5) [10].

Eine Queue ist standardmäßig out-of-order. Sie darf Aufträge in beliebiger Reihenfolge ausführen, solange die angegebenen Abhängigkeiten eingehalten werden. Eine in-order-Queue führt dagegen alle Aufträge streng nacheinander aus. Das ist einfacher, verschenkt aber Parallelität zwischen unabhängigen Aufträgen. Wer Buffer verwendet, muss Abhängigkeiten nicht selbst angeben, weil die Laufzeit sie aus den Accessoren ableitet. Die gröbste Form der Synchronisation ist das Warten des Hosts auf die gesamte Queue. Danach ist sicher jeder bisher eingereichte Auftrag beendet.

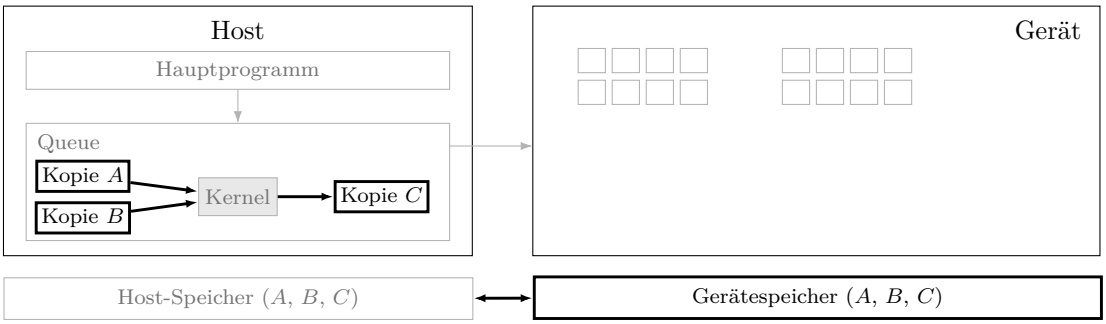


Abbildung 5: Getrennter Speicher und Abhängigkeiten (Stufe 2). Neu sind der Gerätespeicher und explizite Kopien, deren Abhängigkeiten als Pfeile in der Queue stehen. Kopie A und Kopie B hängen nicht voneinander ab und können gleichzeitig laufen

5.4 Hierarchische Ausführung und lokaler Speicher

Im einfachen Indexraum aus Abschnitt 5.1 sind alle Work-Items gleichberechtigt und können nicht zusammenarbeiten. Für Optimierungen erlaubt SYCL deshalb einen zweistufigen Indexraum, die nd_range. Sie besteht aus einem globalen Bereich und einem lokalen Bereich, der den globalen in gleich große Work-Groups aufteilt (Abbildung 6). Der globale Bereich muss dafür in jeder Dimension ein Vielfaches des lokalen sein. Jedes Work-Item kennt nun drei Positionen, seine globale ID im gesamten Indexraum, seine lokale ID innerhalb der Work-Group und die ID seiner Work-Group.

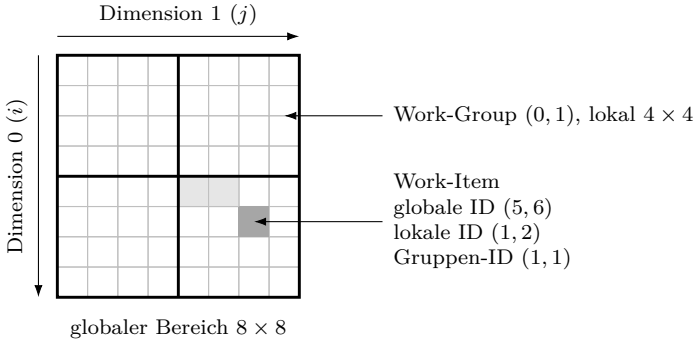


Abbildung 6: Eine nd_range mit globalem Bereich 8×8 und lokalem Bereich 4×4. Dick umrandet sind die vier Work-Groups

Die Work-Items einer Work-Group laufen auf derselben Recheneinheit und teilen sich deren lokalen Speicher (Abbildung 4). Sie können damit zusammenarbeiten. Typischerweise lädt jedes Work-Item einen Teil der Daten aus dem globalen in den lokalen Speicher, und anschließend lesen alle Work-Items die gemeinsam geladenen Werte. Damit kein Work-Item einen Wert liest, bevor ein anderes ihn geschrieben hat, synchronisieren sie sich an einer Barriere. Jedes Work-Item wartet dort, bis alle Work-Items seiner Gruppe sie erreicht haben, und erst danach sind die Schreibzugriffe der anderen sichtbar. Eine Barriere über Work-Groups hinweg gibt es nicht. Work-Groups laufen

unabhängig voneinander und in beliebiger Reihenfolge, weshalb sie nur über das Ende eines Kernels synchronisiert werden können.

Lokaler Speicher wird nicht im Kernel angelegt, sondern vor dem Start reserviert, da die Laufzeit seinen Bedarf kennen muss, bevor sie die Work-Groups auf die Recheneinheiten verteilt. Der Host reicht den Kernel dazu zusammen mit dieser Reservierung als Command Group ein (Abbildung 7). Eine Command Group bündelt einen Auftrag mit allem, was er braucht, also Speicher, Accessoren und Abhängigkeiten. Die Kurzformen aus den vorigen Abschnitten sind nur eine Abkürzung dafür.

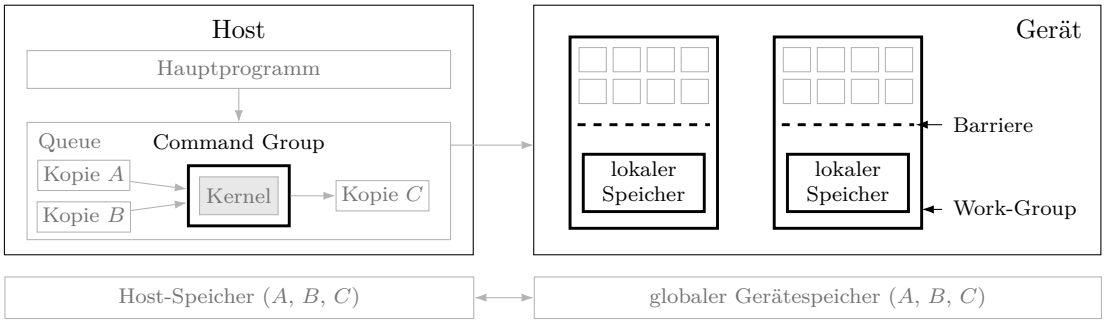


Abbildung 7: Hierarchische Ausführung (Stufe 3). Neu sind die Command Group sowie Work-Groups mit eigenem lokalem Speicher und Barriere

5.5 Abbildung auf die Hardware

Die Hierarchie aus Indexraum, Work-Group und Work-Item ist abstrakt. Wie sie auf ein Gerät abgebildet wird, entscheidet das Backend (Abbildung 8). Auf einer NVIDIA-GPU wird eine Work-Group zu einem Thread-Block, der auf einem Streaming-Multiprozessor läuft, und ein Work-Item zu einem Thread. Der lokale Speicher entspricht dem Shared Memory des Multiprozessors. Das Modell entspricht damit weitgehend CUDA [1], was die Migration bestehender Anwendungen erleichtert. Auf der CPU übernimmt ein Kern eine Work-Group und verteilt deren Work-Items auf seine Vektoreinheit (SIMD, ein Befehl für mehrere Werte). Der lokale Speicher liegt dort im gewöhnlichen Arbeitsspeicher und profitiert vor allem von den Caches.

Zwischen Work-Group und Work-Item liegt die Sub-Group. Sie umfasst die Work-Items, die im Gleichschritt dieselben Befehle ausführen. Auf der GPU entspricht sie einem Warp aus 32 Threads, auf der CPU einem SIMD-Vektor. Dieselbe Work-Group kann also auf verschiedenen Geräten sehr unterschiedlich ausgeführt werden. Wie groß eine Work-Group sinnvollerweise ist, hängt deshalb vom Gerät ab.

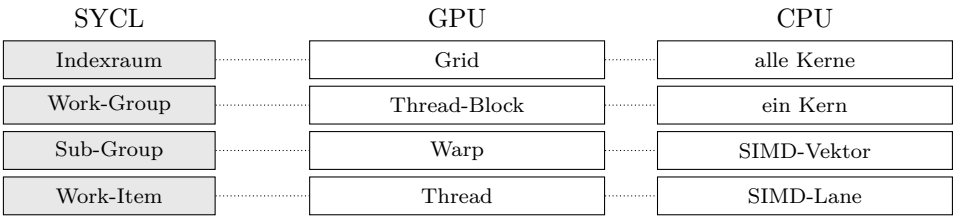


Abbildung 8: Abbildung der SYCL-Ausführungshierarchie auf GPU und CPU

5.6 Geräteauswahl, Backends und JIT-Kompilierung

Eine SYCL-Implementierung spricht jedes Gerät über ein Backend an, etwa CUDA für NVIDIA-GPUs, HIP für AMD-GPUs, Level Zero für Intel-GPUs oder OpenCL für CPUs. Die Laufzeit fasst die Geräte eines Backends zu Plattformen zusammen. Welches Gerät eine Queue nutzt, bestimmt ein Device Selector. Das ist eine Funktion, die jedem verfügbaren Gerät eine Bewertung gibt, und das Gerät mit der höchsten Bewertung wird gewählt. Der Standard liefert Selektoren für die Standardauswahl, für GPUs und für CPUs. Eigene Selektoren können beliebige Kriterien heranziehen. Zusätzlich lässt sich die Menge der sichtbaren Geräte bei DPC++ über eine Umgebungsvariable einschränken, ohne das Programm zu ändern.

Damit dasselbe Programm ohne Neuübersetzung auf verschiedenen Geräten laufen kann, enthält die ausführbare Datei neben dem Host-Code den Gerätecode in einer oder mehreren Zwischendarstellungen, etwa SPIR-V für OpenCL oder PTX für CUDA (Abbildung 9). Erst beim ersten Start eines Kernels übersetzt der Treiber die passende Zwischendarstellung just in time für das konkrete Gerät [7]. Der erste Lauf ist deshalb langsamer als alle folgenden. Alternativ kann der Gerätecode vorab für ein bekanntes Gerät übersetzt werden, was den Start beschleunigt, aber die Portabilität des Programms einschränkt.

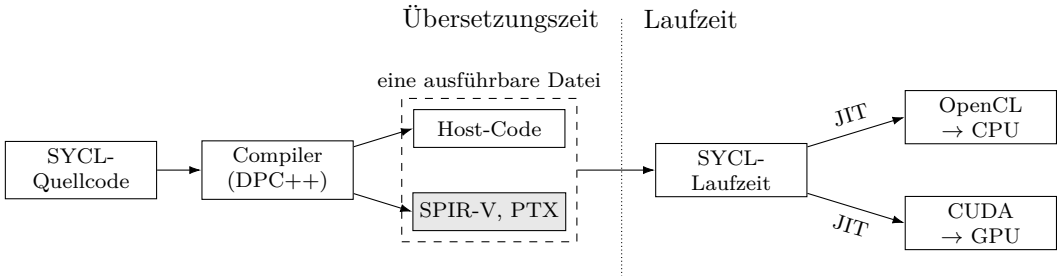


Abbildung 9: Übersetzung und Ausführung. Der Gerätecode liegt als Zwischendarstellung vor und wird erst beim ersten Start für das gewählte Gerät übersetzt

Über Geräteabfragen liest ein Programm zur Laufzeit Eigenschaften des gewählten Geräts aus, zum Beispiel seinen Namen, die Zahl der Recheneinheiten, die Größe des lokalen Speichers oder die maximale Zahl von Work-Items je Work-Group. So kann es

Parameter wie die Work-Group-Größe an das Gerät anpassen, statt sie fest vorzugeben [3].

5.7 Fehlerbehandlung

Weil Host und Gerät asynchron arbeiten, unterscheidet SYCL zwei Arten von Fehlern (Abbildung 10). Synchrone Fehler erkennt die Laufzeit schon beim Aufruf auf dem Host, etwa eine Work-Group, die größer ist, als das Gerät erlaubt. Sie werden sofort als gewöhnliche C++-Ausnahme geworfen. Asynchrone Fehler treten erst während der Ausführung auf dem Gerät auf, wenn der Host längst weitergearbeitet hat. Die Laufzeit sammelt sie und übergibt sie erst dann an eine vom Programm registrierte Funktion, den Async Handler, wenn der Host ausdrücklich darauf wartet. Ohne eigenen Handler beendet die Implementierung das Programm in der Regel. Ein robustes Programm muss deshalb beide Wege behandeln [3].

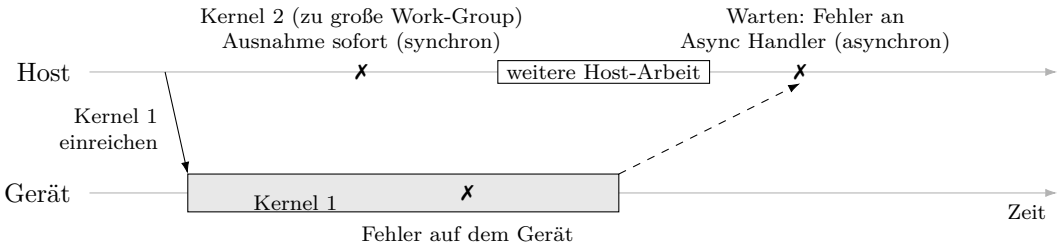


Abbildung 10: Synchrone und asynchrone Fehler. Der synchrone Fehler wird beim Einreichen gemeldet, der asynchrone erst, wenn der Host auf das Gerät wartet

5.8 Zusammenfassung

Tabelle 4 ordnet die Konzepte den drei Problemen aus Abschnitt 1 und den Stufen der Portierung zu. Die einfachen Konzepte genügen für ein lauffähiges, portables Programm. Die weiteren geben schrittweise Kontrolle über Speicher und Ausführung zurück.

Konzept	adressiertes Problem	Stufe
Single-Source in Standard-C++	mehrfache Codebasen	1
Queue, Kernel, Indexraum, USM shared	Abstraktion (einfacher Einstieg)	1
USM device, Kopien, Events, Abhängigkeiten	Kontrolle über Datentransfer	2
Buffer und Accessoren	Abstraktion (Abhängigkeiten automatisch)	–
nd_range, Work-Group, lokaler Speicher, Barriere	Kontrolle über Ausführung	3
Backends, Device Selector, JIT-Kompilierung	Herstellerbindung	4
Geräteabfragen, Fehlerbehandlung	Anpassung an das Gerät	4

Tabelle 4: Konzepte von SYCL, die von ihnen adressierten Probleme und die Stufe, in der sie bei der Portierung eingeführt werden

6 Anwendung auf die Matrixmultiplikation

Dieses Kapitel zeigt, wie die Konzepte aus Abschnitt 5 im Programm zur Matrixmultiplikation umgesetzt sind. Der vollständige Quellcode aller Stufen steht in Anhang A. Verweise der Form (A.2, Z. 36–43) bezeichnen den Abschnitt des Anhangs und die Zeilen im dortigen Listing. Alle Stufen nutzen dieselben Hilfsfunktionen für Zeitmessung, Ergebnisprüfung und Ausgabe (A.5, Z. 1–90), sodass sich die Programme nur in den hier beschriebenen Stellen unterscheiden.

6.1 Stufe 0 – Ausgangsprogramm

Das Ausgangsprogramm berechnet $C = A \cdot B$ mit drei verschachtelten Schleifen (A.1, Z. 12–20). Die äußeren Schleifen über i und j durchlaufen alle Elemente von C, die innere Schleife über k bildet das Skalarprodukt aus Zeile i von A und Spalte j von B. Die Matrizen liegen zeilenweise in eindimensionalen Feldern, das Element (i, j) also an Position $i \cdot n + j$. Die Berechnungen der einzelnen Elemente von C sind voneinander unabhängig. Genau diese Unabhängigkeit nutzen die folgenden Stufen aus. Der Messablauf mit einem getrennt erfassten ersten Lauf und fünf Messläufen (A.1, Z. 30–40) ist in allen Stufen gleich.

6.2 Stufe 1 – Der erste Kernel

Stufe 1 ersetzt die beiden äußeren Schleifen durch einen Indexraum und lässt den Rest des Programms möglichst unverändert. Tabelle 5 listet die Änderungen gegenüber Stufe 0.

Die Queue wird ohne Argument angelegt (A.2, Z. 24). Sie nutzt damit den Standard-

Konzept	Umsetzung	Stelle
Queue, Geräteauswahl	Queue mit Standard-Selektor anlegen	(A.2, Z. 24)
Geräteabfrage	Gerätenamen für die Ausgabe auslesen	(A.2, Z. 25)
USM shared	A, B, C mit `malloc_shared` anlegen	(A.2, Z. 28–30)
Kernel, Indexraum	`parallel_for` über $n \times n$ ersetzt die i- und j-Schleife	(A.2, Z. 36–43)
Synchronisation	Host wartet auf das Ende des Kernels	(A.2, Z. 43)
Speicherfreigabe	Freigabe über die Queue	(A.2, Z. 63–65)

Tabelle 5: Änderungen in Stufe 1 gegenüber Stufe 0

Selektor, der das Gerät aus den sichtbaren Geräten wählt. Welche das sind, wird in Stufe 4 von außen gesteuert. Da die Matrizen in USM shared liegen, kann der Host sie wie gewöhnliche Felder befüllen und das Ergebnis direkt prüfen (A.2, Z. 31–32, 60), ohne dass eine Zeile für den Datentransfer nötig ist.

Den Kern der Änderung zeigt Listing 1. Der Indexraum hat die Form der Ergebnismatrix, und jedes der n^2 Work-Items berechnet genau ein Element von C (Abbildung 11). Die beiden Komponenten der ID übernehmen die Rolle der früheren Schleifenvariablen i und j. Die innere Schleife über k bleibt unverändert im Kernel. Der Lambda-Ausdruck übernimmt die Zeiger A, B, C und die Größe n als Kopie, was bei USM genügt, weil die Zeiger auch auf dem Gerät gültig sind. Am Ende wartet der Host auf den Kernel, da er das Ergebnis anschließend misst und prüft.

Listing 1: Kernel in Stufe 1 (Ausschnitt aus Anhang A.2)

```
36    q.parallel_for(sycl::range<2>{n, n}, [=](sycl::id<2> idx) {
37       const std::size_t i = idx[0];
38       const std::size_t j = idx[1];
39       float sum = 0.0f;
40       for (std::size_t k = 0; k < n; ++k)
41         sum += A[i * n + k] * B[k * n + j];
42       C[i * n + j] = sum;
43     }).wait();  // auf das Ende des Kernels warten
```

6.3 Stufe 2 – Expliziter Gerätespeicher

Stufe 2 ändert nur die Speicherverwaltung, der Kernel ist identisch mit Stufe 1 (A.3, Z. 43–50). Die Matrizen liegen nun zweimal vor, als gewöhnliche C++-Vektoren auf dem Host (A.3, Z. 28–30) und als USM-device-Speicher auf dem Gerät (A.3, Z. 33–35). Vor dem Kernel kopiert der Host A und B auf das Gerät (A.3, Z. 37–41), danach C zurück (A.3, Z. 52). Die Prüfung des Ergebnisses greift nun auf die Host-Kopie zu (A.3, Z. 77).

Die beiden Kopien von A und B werden ohne gegenseitige Abhängigkeit eingereicht

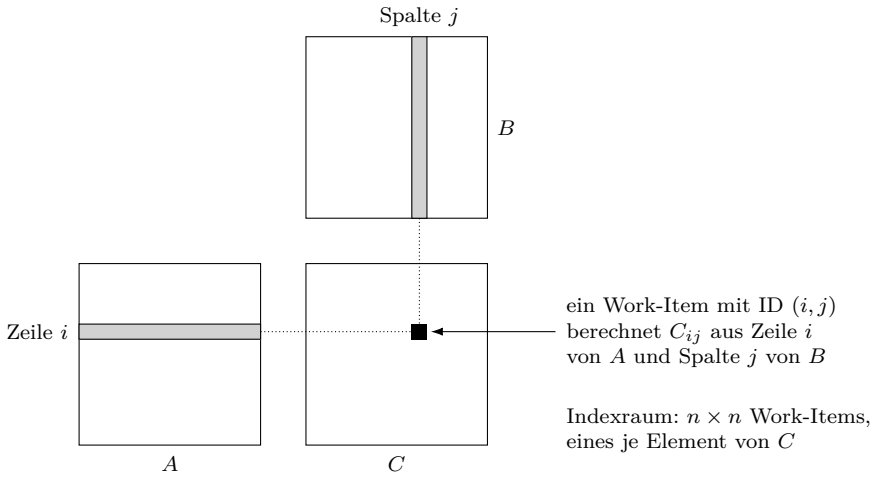


Abbildung 11: Zuordnung der Work-Items zu den Elementen von C in Stufe 1

und können daher gleichzeitig laufen, wie in Abbildung 5 dargestellt. Die Abhängigkeit des Kernels von den Kopien ist im Programm allerdings nicht über Events ausgedrückt. Der Host wartet stattdessen nach jedem Abschnitt auf die gesamte Queue (A.3, Z. 40, 50, 52). Das ist die gröbste Form der Synchronisation aus Abschnitt 5.3. Sie ist hier bewusst gewählt, weil nur so Kopier- und Rechenzeit getrennt gemessen werden können (A.3, Z. 61–75). Ein Programm, das nicht misst, würde den Kernel stattdessen von den Events der beiden Kopien abhängig machen und so Transfer und Rechnung anderer Aufträge überlappen lassen. Buffer und Accessoren werden hier nicht verwendet, da USM dem Ausgangsprogramm mit seinen Zeigern am nächsten ist.

6.4 Stufe 3 – Kachelung im lokalen Speicher

In den Stufen 1 und 2 liest jedes Work-Item eine ganze Zeile von A und eine ganze Spalte von B aus dem globalen Speicher, insgesamt $2n$ Werte. Benachbarte Work-Items lesen dabei dieselben Werte immer wieder. Stufe 3 nutzt deshalb Work-Groups und lokalen Speicher (Abschnitt 5.4). Die Matrizen werden in Kacheln zerlegt, quadratische Teilblöcke der Größe $T \times T$, und jede Work-Group aus $T \times T$ Work-Items berechnet eine Kachel von C (Abbildung 12). Dazu lädt sie in n/T Schritten je eine Kachel von A und B in den lokalen Speicher und rechnet mit diesen Werten weiter. Jedes Work-Item lädt pro Schritt nur zwei Werte, insgesamt also $2n/T$ statt $2n$ Werte aus dem globalen Speicher.

Der Kernel wird nun als Command Group eingereicht (A.4, Z. 60), da er lokalen Speicher braucht. Darin werden zwei lokale Felder der Größe $T \times T$ für die Kacheln von A und B reserviert (A.4, Z. 62–63). Statt eines einfachen Indexraums erhält der Kernel eine nd_range mit dem globalen Bereich $n_p \times n_p$ und dem lokalen Bereich $T \times T$ (A.4, Z. 66). Da der globale Bereich ein Vielfaches des lokalen sein muss, wird n zuvor auf das

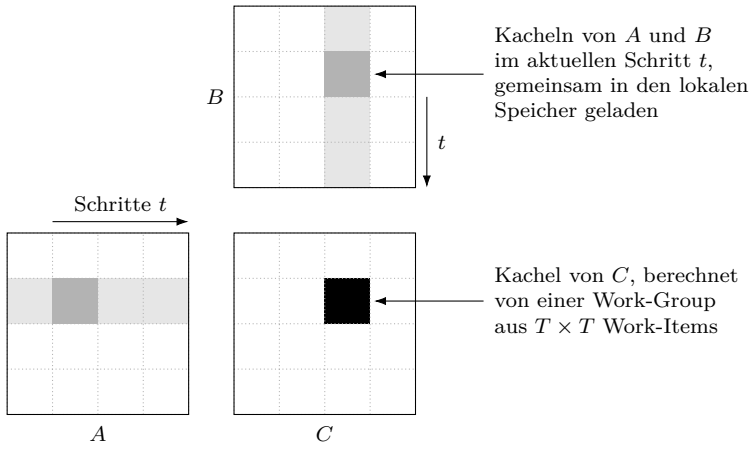


Abbildung 12: Kachelung in Stufe 3. Eine Work-Group durchläuft die Kachelzeile von A und die Kachelspalte von B in n/T Schritten

nächste Vielfache von T aufgerundet (A.4, Z. 51). Jedes Work-Item liest seine globale ID als Position in C und seine lokale ID als Position in der Kachel (A.4, Z. 68–71). Die Speicherverwaltung ist unverändert aus Stufe 2 übernommen (A.4, Z. 45–57, 90).

Listing 2 zeigt die Schleife über die Kacheln. Jedes Work-Item lädt ein Element der aktuellen Kachel von A und von B in den lokalen Speicher (Zeilen 76–77). Die erste Barriere (Zeile 78) stellt sicher, dass beide Kacheln vollständig geladen sind, bevor ein Work-Item mit ihnen rechnet. Die innere Schleife (Zeilen 81–82) liest dann nur noch aus dem lokalen Speicher. Die zweite Barriere (Zeile 83) verhindert, dass ein schnelles Work-Item im nächsten Schritt die Kachel überschreibt, während andere noch mit ihr rechnen. Elemente außerhalb der Matrix werden beim Laden durch null ersetzt und beim Schreiben ausgelassen (A.4, Z. 85), sodass auch Matrixgrößen funktionieren, die kein Vielfaches von T sind.

Listing 2: Schleife über die Kacheln in Stufe 3 (Ausschnitt aus Anhang A.4)

```
73            float sum = 0.0f;
74            for (std::size_t t = 0; t < n; t += T) {
75              // gemeinsam eine Kachel laden (außerhalb der Matrix: 0)
76              tileA[li][lj] = (i < n && t + lj < n) ? A[i * n + t + lj] : 0.0f;
77              tileB[li][lj] = (t + li < n && j < n) ? B[(t + li) * n + j] : 0.0f;
78              sycl::group_barrier(it.get_group());
79
80              // mit den Daten aus dem schnellen Local Memory rechnen
81              for (std::size_t k = 0; k < T; ++k)
82                sum += tileA[li][k] * tileB[k][lj];
83              sycl::group_barrier(it.get_group());
84            }
85            if (i < n && j < n) C[i * n + j] = sum;
```

6.5 Stufe 4 – Gerätewechsel und Kachelgröße

Stufe 4 enthält keinen neuen Code, sondern führt die Stufen 1 bis 3 unverändert auf anderen Geräten aus. Übersetzt wird jedes Programm einmal mit DPC++ für zwei Zwischendarstellungen, PTX für die NVIDIA-GPUs und SPIR-V für die CPU über OpenCL (Abbildung 9). Das Gerät wird beim Start über die Umgebungsvariable `ONEAPI_DEVICE_SELECTOR` gewählt, etwa `cuda:gpu` oder `opencl:cpu`. Sie schränkt die sichtbaren Geräte ein, aus denen der Standard-Selektor der Queue dann wählt (A.4, Z. 38). Der per Geräteabfrage gelesene Gerätename (A.4, Z. 39) landet in der Ausgabe jeder Messung. So ist jedem Messwert das Gerät zugeordnet, auf dem er entstand.

Variiert wird außerdem die Kachelgröße $T \in \{8, 16, 32, 64\}$ aus Stufe 3. Sie ist eine Konstante, die beim Übersetzen gesetzt wird (A.4, Z. 28–30, 36), sodass für jede Kachelgröße ein eigenes Programm entsteht. Bei $T = 64$ umfasst eine Work-Group 4096 Work-Items und überschreitet die GPU-Grenze von 1024 Threads je Thread-Block [15]. Die Laufzeit erkennt das beim Einreichen der nd_range (A.4, Z. 60–87) und wirft einen synchronen Fehler (Abschnitt 5.7). Da das Programm keine Fehlerbehandlung enthält, bricht das Programm ab, und das Messskript vermerkt den Lauf als „startet nicht“. Auf der CPU ist die erlaubte Work-Group-Größe höher, dort läuft $T = 64$ fehlerfrei.

Abhilfe würde eine Geräteabfrage der maximalen Work-Group-Größe schaffen, nach der das Programm T zur Laufzeit wählt (Abschnitt 5.6). Das Programm verzichtet bewusst darauf, damit derselbe Wert von T auf allen Geräten verglichen werden kann. Die Kachelgröße bleibt damit ein Parameter, der für jedes Gerät von Hand gewählt werden muss, und ist zugleich der einzige, der zwischen den Geräten variiert wird. Ihr Einfluss auf Leistung und Performance-Portabilität wird in Abschnitt 7 ausgewertet.

7 Ergebnisse

Alle Stufen liefen ohne Codeänderung und mit korrektem Ergebnis auf allen drei Geräten. Tabelle 6 zeigt die Wirkung jeder Stufe. Schon Stufe 1 ist bei $n = 1024$ rund 2300-mal (RTX) bzw. 10 700-mal (L40S) schneller als das sequentielle Ausgangsprogramm auf einem CPU-Kern. Dieser Vergleich misst allerdings Parallelisierung und Gerätewechsel zugleich. Stufe 2 ändert die Rechenzeit nicht, macht aber den Transfer sichtbar, der bei $n = 2048$ ein Viertel (RTX) bis ein Drittel (L40S) der Rechenzeit ausmacht. Die Kachelung in Stufe 3 steigert die Leistung um 31 % auf der RTX, auf der L40S sogar um 69 %. Auf der CPU erreicht Stufe 1 mit sechs Kernen rund 45 GFLOPS, etwa 90-mal so viel wie ein einzelner Kern. Die Kachelung steigert die Leistung dort auf mehr als das Dreifache, weil die Kacheln in die Caches der CPU passen. Der erste Lauf mit JIT-Kompilierung dauert bei $n = 1024$ zwei- bis achtmal so lang wie die folgenden.

	RTX 5060 Ti		**L40S**		**Ryzen 5 5600X**	
Stufe	$n = 1024$	$n = 2048$	$n = 1024$	$n = 2048$	$n = 1024$	$n = 2048$
0 (ein CPU-Kern)	0,5	–	0,3	–	0,5	–
1 (USM shared)	1105	1099	3459	2929	45	43
2 (USM device)	1088	1083	3402	2926	44	43
3 (Kachelung, $T = 16$)	1395	1418	4720	4934	154	150

Tabelle 6: Leistung je Stufe in GFLOPS (Median aus fünf Läufen, Stufe 0 auf einem Kern der jeweiligen Host-CPU)

	RTX 5060 Ti		**L40S**		**Ryzen 5 5600X**	
Stufe	$n = 1024$	$n = 2048$	$n = 1024$	$n = 2048$	$n = 1024$	$n = 2048$
0 (ein CPU-Kern)	4484	–	6650	–	4484	–
1 (USM shared)	1,94	15,64	0,62	5,87	47,6	401
2 Kernel	1,97	15,87	0,63	5,87	48,7	402
Transfer	1,34	3,93	0,60	2,12	0,90	5,68
3 Kernel ($T = 16$)	1,54	12,12	0,46	3,48	14,0	114
Transfer	1,40	4,21	0,73	2,15	1,05	5,74

Tabelle 7: Laufzeit je Stufe in ms (Median aus fünf Läufen). Ab Stufe 2 sind Kernel- und Transferzeit getrennt erfasst

Tabelle 7 zeigt die zugehörigen Laufzeiten. Bei kleinen Matrizen kostet der Datentransfer zur GPU mehr als die Rechnung. Auf der RTX überwiegt die Rechenzeit ab $n = 1024$, auf der schnelleren L40S erst bei $n = 2048$. Auf der CPU ist der Transfer vernachlässigbar, da Host und Gerät denselben Arbeitsspeicher nutzen. Abbildung 13 zeigt die Effizienz je Kachelgröße auf allen drei Geräten. Die beste Kachelgröße ist geräteabhängig. Auf beiden GPUs ist $T = 16$ am schnellsten, auf der CPU $T = 32$. Bei

$T = 64$ startet der Kernel auf den GPUs nicht, sodass die Performance-Portabilität (PP) auf null fällt. Nach Pennycook et al. [13] ist sie das harmonische Mittel der Effizienzen, die eine Anwendung auf den einzelnen Geräten einer Gerätemenge erreicht. Wird ein Gerät nicht unterstützt, ist sie null. Als Effizienz dient hier die Leistung relativ zur besten Kachelgröße auf demselben Gerät. Den höchsten Wert von 0,99 erreicht $T = 32$.

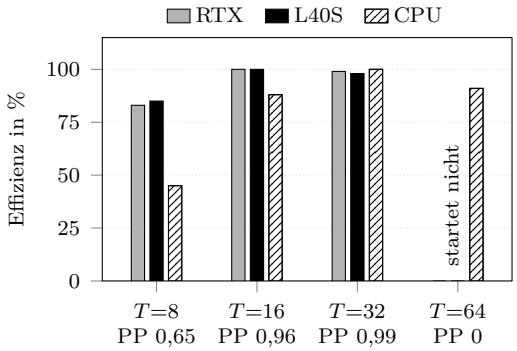


Abbildung 13: Effizienz und Performance-Portabilität je Kachelgröße bei $n = 2048$

8 Diskussion

Für F1 zeigen die Stufen, welches Konzept welches Problem löst (Tabelle 4). Die Herstellerbindung lösen Backends und Geräteauswahl (Stufe 4), die mehrfachen Codebasen Single-Source in Standard-C++ (Stufe 1). Den Zielkonflikt zwischen Abstraktion und Kontrolle entschärft SYCL durch einen gestuften Einstieg. Ein einfacher Indexraum mit USM shared genügt für ein lauffähiges Programm, während Work-Groups, lokaler Speicher und expliziter Transfer die Kontrolle für die Optimierung zurückgeben (Stufen 2 und 3).

Für F2 bestätigt die Untersuchung funktionale Portabilität für alle Stufen. Performance-Portabilität ergibt sich dagegen nicht von selbst. SYCL abstrahiert die Schnittstelle, nicht die Hardware. Parameter wie die Work-Group-Größe bleiben gerätespezifisch, und ein für die CPU günstiger Wert kann auf der GPU sogar zum Abbruch führen. Dies deckt sich mit der in früheren Studien beobachteten Abhängigkeit von Implementierung und Parametrierung [11]. Selbst zwischen zwei NVIDIA-GPUs unterscheidet sich der Nutzen der Kachelung deutlich (31 % gegenüber 69 %), auch wenn die beste Kachelgröße übereinstimmt. Den größten Gewinn bringt bereits der erste, unveränderte Kernel. Die hardwarenahe Kachelung lohnt sich zusätzlich, bindet das Programm aber an gerätespezifische Parameter.

Die Aussagekraft der Untersuchung ist begrenzt. Untersucht wurden ein einzelner Kernel, drei Geräte mit GPUs nur eines Herstellers und eine Implementierung. Die

PP-Werte gelten nur für diese Gerätemenge [16], und ein Vergleich mit nativem CUDA-Code oder optimierten Bibliotheken fehlt.

9 Schlussfolgerung

SYCL erfüllt sein Versprechen „einmal schreiben, überall ausführen“ für die Korrektheit. Ein schrittweise portiertes Programm lief in allen Stufen unverändert auf CPU und GPU. Die gerätespezifische Abstimmung für hohe Leistung nimmt SYCL dem Programmierer jedoch nicht ab. Folgearbeiten sollten GPUs weiterer Hersteller, etwa die AMD MI300A des DACHS-Clusters, und eine zweite Implementierung wie AdaptiveCpp einbeziehen.

Literaturverzeichnis

[1] J. Nickolls, I. Buck, M. Garland und K. Skadron, „Scalable Parallel Programming with CUDA," *ACM Queue*, Jg. 6, Nr. 2, S. 40–53, 2008. DOI: 10.1145/1365490.1365500.

[2] J. E. Stone, D. Gohara und G. Shi, „OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems," *Computing in Science & Engineering*, Jg. 12, Nr. 3, S. 66–73, 2010. DOI: 10.1109/MCSE.2010.69.

[3] The Khronos SYCL Working Group. „SYCL 2020 Specification," besucht am 30. Okt. 2026. Adresse: https://registry.khronos.org/SYCL/specs/sycl-2020/html/sycl-2020.html.

[4] Intel Corporation. „Intel LLVM-based Compiler (DPC++) – Repository," besucht am 30. Okt. 2026. Adresse: https://github.com/intel/llvm.

[5] A. Alpay und V. Heuveline, „SYCL beyond OpenCL: The Architecture, Current State and Future Direction of hipSYCL," in *Proceedings of the International Workshop on OpenCL (IWOCL '20)*, 2020. DOI: 10.1145/3388333.3388658.

[6] AdaptiveCpp Contributors. „AdaptiveCpp – Repository," besucht am 30. Okt. 2026. Adresse: https://github.com/AdaptiveCpp/AdaptiveCpp.

[7] A. Alpay und V. Heuveline, „One Pass to Bind Them: The First Single-Pass SYCL Compiler with Unified Code Representation Across Backends," in *Proceedings of the International Workshop on OpenCL (IWOCL '23)*, 2023.

[8] H. C. Edwards, C. R. Trott und D. Sunderland, „Kokkos: Enabling Manycore Performance Portability through Polymorphic Memory Access Patterns," *Journal of Parallel and Distributed Computing*, Jg. 74, Nr. 12, S. 3202–3216, 2014. DOI: 10.1016/j.jpdc.2014.07.003.

[9] D. A. Beckingsale u. a., „RAJA: Portable Performance for Large-Scale Scientific Applications," in *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, 2019, S. 71–81.

[10] J. Reinders, B. Ashbaugh, J. Brodman, M. Kinsner, J. Pennycook und X. Tian, *Data Parallel C++: Programming Accelerated Systems Using C++ and SYCL*, 2. Aufl. Berkeley, CA: Apress, 2023, ISBN: 978-1-4842-9691-2. DOI: 10.1007/978-1-4842-9691-2.

[11] T. Deakin und S. McIntosh-Smith, „Evaluating the Performance of HPC-Style SYCL Applications," in *Proceedings of the International Workshop on OpenCL (IWOCL '20)*, 2020. DOI: 10.1145/3388333.3388643.

[12] M. Breyer, A. Van Craen und D. Pflüger, „A Comparison of SYCL, OpenCL, CUDA, and OpenMP for Massively Parallel Support Vector Machine Classification on Multi-Vendor Hardware," in *Proceedings of the International Workshop on OpenCL (IWOCL '22)*, 2022.

[13] S. J. Pennycook, J. D. Sewall und V. W. Lee, „A Metric for Performance Portability," in *Proceedings of the 7th International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2016. arXiv: 1611.07409.

[14] W.-m. W. Hwu, D. B. Kirk und I. El Hajj, *Programming Massively Parallel Processors: A Hands-on Approach*, 4. Aufl. Cambridge, MA: Morgan Kaufmann, 2022.

[15] NVIDIA Corporation. „CUDA C++ Programming Guide," besucht am 30. Okt. 2026. Adresse: `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`.

[16] S. J. Pennycook, J. D. Sewall und V. W. Lee, „Implications of a Metric for Performance Portability," *Future Generation Computer Systems*, Jg. 92, S. 947–958, 2019.

A Quellcode der Matrixmultiplikation

Die folgenden Listings zeigen den vollständigen Quellcode der Stufen 0 bis 3. Stufe 4 verwendet dieselben Programme. Die Kachelgröße von Stufe 3 wird beim Übersetzen mit `-DTILE=8|16|32|64` gesetzt, das Gerät über `ONEAPI_DEVICE_SELECTOR` gewählt.

A.1 Stufe 0 – Sequentielles Ausgangsprogramm

```
// Stufe 0 - Das "existierende Programm"
//
// Klassische Matrixmultiplikation C = A * B in reinem C++, ein Kern,
// keine Parallelisierung. Dient als Ausgangspunkt und Referenz für
// Laufzeit und Korrektheit.
//
// Bild für die Folie: drei verschachtelte Schleifen. Jedes Element C[i][j]
// ist das Skalarprodukt aus Zeile i von A und Spalte j von B.

#include "common.hpp"

static void matmul(const float* A, const float* B, float* C, std::size_t n) {
  for (std::size_t i = 0; i < n; ++i)          // jede Zeile von C
    for (std::size_t j = 0; j < n; ++j) {      // jede Spalte von C
      float sum = 0.0f;
      for (std::size_t k = 0; k < n; ++k)      // Skalarprodukt
        sum += A[i * n + k] * B[k * n + j];
      C[i * n + j] = sum;
    }
}

int main(int argc, char** argv) {
  const auto cfg = bench::parse_args(argc, argv);
  const std::size_t n = cfg.n;

  std::vector<float> A(n * n), B(n * n), C(n * n);
  bench::fill_random(A.data(), n * n, 1);
  bench::fill_random(B.data(), n * n, 2);

  // Erster Lauf (zum Vergleich mit dem "ersten Lauf" der SYCL-Stufen)
  bench::Timer t0;
  matmul(A.data(), B.data(), C.data(), n);
  const double first = t0.ms();

  std::vector<double> times;
  for (int r = 0; r < cfg.reps; ++r) {
    bench::Timer t;
    matmul(A.data(), B.data(), C.data(), n);
    times.push_back(t.ms());
  }

  const bool ok = bench::verify(A.data(), B.data(), C.data(), n);
  bench::report("0_sequentiell", "CPU (1 Kern)", n, first, bench::median(times), 0.0, ok);
  return ok ? 0 : 1;
}
```

A.2 Stufe 1 – USM shared und `parallel_for`

```
// Stufe 1 - Der erste SYCL-Kernel
//
// Kleinstmögliche Änderung gegenüber Stufe 0:
//   * eine Queue anlegen (sie wählt das Gerät, z. B. die GPU)
//   * Speicher mit malloc_shared anlegen (USM "shared"): derselbe Zeiger
//     ist auf Host und Gerät gültig, die Laufzeit verschiebt die Daten
//     automatisch dorthin, wo sie gebraucht werden
//   * die beiden äußeren Schleifen (i, j) durch parallel_for ersetzen -
//     jedes C[i][j] wird von einem eigenen Work-Item berechnet
//
// Die innere Schleife (k) bleibt unverändert im Kernel.
//
// Neue Konzepte: queue, device, kernel, parallel_for, range/id, USM shared

#include <sycl/sycl.hpp>
#include "common.hpp"

int main(int argc, char** argv) {
  const auto cfg = bench::parse_args(argc, argv);
  const std::size_t n = cfg.n;

  // 1) Queue: Verbindung zu einem Gerät. Ohne Angabe nimmt SYCL das
  //    "beste" verfügbare Gerät (meist die GPU).
  sycl::queue q;
  const std::string dev = q.get_device().get_info<sycl::info::device::name>();

  // 2) Gemeinsamer Speicher (Unified Shared Memory, Variante "shared")
  float* A = sycl::malloc_shared<float>(n * n, q);
  float* B = sycl::malloc_shared<float>(n * n, q);
  float* C = sycl::malloc_shared<float>(n * n, q);
  bench::fill_random(A, n * n, 1);
  bench::fill_random(B, n * n, 2);

  // 3) Kernel: ersetzt die i- und j-Schleife aus Stufe 0
  auto run = [&] {
    q.parallel_for(sycl::range<2>{n, n}, [=](sycl::id<2> idx) {
       const std::size_t i = idx[0];
       const std::size_t j = idx[1];
       float sum = 0.0f;
       for (std::size_t k = 0; k < n; ++k)
         sum += A[i * n + k] * B[k * n + j];
       C[i * n + j] = sum;
     }).wait();  // auf das Ende des Kernels warten
  };

  // Erster Lauf: enthält ggf. JIT-Kompilierung des Kernels und das erste
  // Verschieben der Daten auf das Gerät -> separat ausweisen!
  bench::Timer t0;
  run();
  const double first = t0.ms();

  std::vector<double> times;
  for (int r = 0; r < cfg.reps; ++r) {
    bench::Timer t;
    run();
    times.push_back(t.ms());
  }
```

```
58
59    // Zugriff auf C vom Host aus: mit malloc_shared einfach möglich
60    const bool ok = bench::verify(A, B, C, n);
61    bench::report("1_usm_shared", dev, n, first, bench::median(times), 0.0, ok);
62
63    sycl::free(A, q);
64    sycl::free(B, q);
65    sycl::free(C, q);
66    return ok ? 0 : 1;
67  }
```

A.3 Stufe 2 – USM device und explizites Kopieren

```
1   // Stufe 2 - Speicher selbst verwalten
2   //
3   // Unterschied zu Stufe 1: Statt malloc_shared (automatisches Verschieben)
4   // wird der Speicher explizit getrennt:
5   //   * Host-Speicher (normales std::vector) für Initialisierung und Prüfung
6   //   * Geräte-Speicher (malloc_device), nur auf dem Gerät gültig
7   //   * Kopieren mit q.memcpy, Host -> Gerät und Gerät -> Host
8   //
9   // Der Kernel selbst ist identisch zu Stufe 1.
10  //
11  // Ziel für die Präsentation: Datentransfer wird sichtbar und messbar.
12  // Gerade bei kleinen Matrizen kostet das Kopieren oft mehr als die Rechnung.
13  //
14  // Neue Konzepte: malloc_device, memcpy, Trennung Host-/Geräte-Speicher
15
16  #include <sycl/sycl.hpp>
17  #include "common.hpp"
18
19  int main(int argc, char** argv) {
20    const auto cfg = bench::parse_args(argc, argv);
21    const std::size_t n = cfg.n;
22    const std::size_t bytes = n * n * sizeof(float);
23
24    sycl::queue q;
25    const std::string dev = q.get_device().get_info<sycl::info::device::name>();
26
27    // Host-Seite
28    std::vector<float> hA(n * n), hB(n * n), hC(n * n);
29    bench::fill_random(hA.data(), n * n, 1);
30    bench::fill_random(hB.data(), n * n, 2);
31
32    // Geräte-Seite
33    float* A = sycl::malloc_device<float>(n * n, q);
34    float* B = sycl::malloc_device<float>(n * n, q);
35    float* C = sycl::malloc_device<float>(n * n, q);
36
37    auto copy_in = [&] {
38      q.memcpy(A, hA.data(), bytes);
39      q.memcpy(B, hB.data(), bytes);
40      q.wait();
41    };
42    auto kernel = [&] {
43      q.parallel_for(sycl::range<2>{n, n}, [=](sycl::id<2> idx) {
44         const std::size_t i = idx[0];
```

```
45        const std::size_t j = idx[1];
46        float sum = 0.0f;
47        for (std::size_t k = 0; k < n; ++k)
48          sum += A[i * n + k] * B[k * n + j];
49        C[i * n + j] = sum;
50      }).wait();
51  };
52  auto copy_out = [&] { q.memcpy(hC.data(), C, bytes).wait(); };
53
54  // Erster Lauf komplett (Kopieren + Kernel inkl. JIT + Zurückkopieren)
55  bench::Timer t0;
56  copy_in();
57  kernel();
58  copy_out();
59  const double first = t0.ms();
60
61  // Messläufe: Kernel und Transfer getrennt erfassen
62  std::vector<double> kernel_times, transfer_times;
63  for (int r = 0; r < cfg.reps; ++r) {
64    bench::Timer tin;
65    copy_in();
66    const double in_ms = tin.ms();
67
68    bench::Timer tk;
69    kernel();
70    kernel_times.push_back(tk.ms());
71
72    bench::Timer tout;
73    copy_out();
74    transfer_times.push_back(in_ms + tout.ms());
75  }
76
77  const bool ok = bench::verify(hA.data(), hB.data(), hC.data(), n);
78  bench::report("2_usm_device", dev, n, first, bench::median(kernel_times),
79                bench::median(transfer_times), ok);
80
81  sycl::free(A, q);
82  sycl::free(B, q);
83  sycl::free(C, q);
84  return ok ? 0 : 1;
85 }
```

A.4 Stufe 3 – Kachelung im lokalen Speicher

```
1  // Stufe 3 - Hardwarenah optimieren: Work-Groups und Local Memory (Tiling)
2  //
3  // Problem der Stufen 1 und 2: Jedes Work-Item liest seine komplette Zeile
4  // von A und Spalte von B aus dem langsamen globalen Speicher. Benachbarte
5  // Work-Items lesen dabei immer wieder dieselben Werte.
6  //
7  // Idee: Die Matrix wird in Kacheln (Tiles) der Größe TILE x TILE zerlegt.
8  //   * Work-Items werden zu Work-Groups von TILE x TILE zusammengefasst
9  //   * Jede Work-Group lädt gemeinsam eine Kachel von A und eine von B in
10 //     den schnellen Local Memory (auf der GPU: Shared Memory des Rechenblocks)
11 //   * Barriere: warten, bis alle Werte geladen sind
12 //   * Jedes Work-Item rechnet mit den Kacheldaten weiter
13 //   * Barriere, dann die nächste Kachel laden
```

```
14 // Dadurch wird jeder Wert aus dem globalen Speicher nur noch einmal pro
15 // Work-Group statt einmal pro Work-Item gelesen.
16 //
17 // Speicherverwaltung wie Stufe 2 - geändert hat sich nur der Kernel.
18 //
19 // Neue Konzepte: nd_range, nd_item, Work-Group, local_accessor (Local
20 // Memory), group_barrier
21 //
22 // Kachelgröße beim Kompilieren ändern, z. B. -DTILE=8 oder -DTILE=32.
23 // Die beste Wahl hängt vom Gerät ab (-> Performance-Portabilität!).
24
25 #include <sycl/sycl.hpp>
26 #include "common.hpp"
27
28 #ifndef TILE
29 #define TILE 16
30 #endif
31
32 int main(int argc, char** argv) {
33   const auto cfg = bench::parse_args(argc, argv);
34   const std::size_t n = cfg.n;
35   const std::size_t bytes = n * n * sizeof(float);
36   constexpr std::size_t T = TILE;
37
38   sycl::queue q;
39   const std::string dev = q.get_device().get_info<sycl::info::device::name>();
40
41   std::vector<float> hA(n * n), hB(n * n), hC(n * n);
42   bench::fill_random(hA.data(), n * n, 1);
43   bench::fill_random(hB.data(), n * n, 2);
44
45   float* A = sycl::malloc_device<float>(n * n, q);
46   float* B = sycl::malloc_device<float>(n * n, q);
47   float* C = sycl::malloc_device<float>(n * n, q);
48
49   // Globaler Bereich auf ein Vielfaches der Kachelgröße aufrunden, damit
50   // auch n, die nicht durch TILE teilbar sind, funktionieren.
51   const std::size_t np = (n + T - 1) / T * T;
52
53   auto copy_in = [&] {
54     q.memcpy(A, hA.data(), bytes);
55     q.memcpy(B, hB.data(), bytes);
56     q.wait();
57   };
58
59   auto kernel = [&] {
60     q.submit([&](sycl::handler& h) {
61        // Local Memory: eine Kachel von A und eine von B pro Work-Group
62        sycl::local_accessor<float, 2> tileA({T, T}, h);
63        sycl::local_accessor<float, 2> tileB({T, T}, h);
64
65        h.parallel_for(
66            sycl::nd_range<2>{{np, np}, {T, T}},  // global, lokal (Work-Group)
67            [=](sycl::nd_item<2> it) {
68              const std::size_t i = it.get_global_id(0);   // Zeile in C
69              const std::size_t j = it.get_global_id(1);   // Spalte in C
70              const std::size_t li = it.get_local_id(0);   // Position in
71              const std::size_t lj = it.get_local_id(1);   // der Kachel
72
```

```
            float sum = 0.0f;
            for (std::size_t t = 0; t < n; t += T) {
              // gemeinsam eine Kachel laden (außerhalb der Matrix: 0)
              tileA[li][lj] = (i < n && t + lj < n) ? A[i * n + t + lj] : 0.0f;
              tileB[li][lj] = (t + li < n && j < n) ? B[(t + li) * n + j] : 0.0f;
              sycl::group_barrier(it.get_group());

              // mit den Daten aus dem schnellen Local Memory rechnen
              for (std::size_t k = 0; k < T; ++k)
                sum += tileA[li][k] * tileB[k][lj];
              sycl::group_barrier(it.get_group());
            }
            if (i < n && j < n) C[i * n + j] = sum;
          });
    }).wait();
  };

  auto copy_out = [&] { q.memcpy(hC.data(), C, bytes).wait(); };

  bench::Timer t0;
  copy_in();
  kernel();
  copy_out();
  const double first = t0.ms();

  std::vector<double> kernel_times, transfer_times;
  for (int r = 0; r < cfg.reps; ++r) {
    bench::Timer tin;
    copy_in();
    const double in_ms = tin.ms();

    bench::Timer tk;
    kernel();
    kernel_times.push_back(tk.ms());

    bench::Timer tout;
    copy_out();
    transfer_times.push_back(in_ms + tout.ms());
  }

  const bool ok = bench::verify(hA.data(), hB.data(), hC.data(), n);
  bench::report("3_tiling_T" + std::to_string(T), dev, n, first,
                bench::median(kernel_times), bench::median(transfer_times), ok);

  sycl::free(A, q);
  sycl::free(B, q);
  sycl::free(C, q);
  return ok ? 0 : 1;
}
```

A.5 Gemeinsame Hilfsfunktionen (`common.hpp`)

```
// common.hpp - gemeinsame Hilfsfunktionen für alle Stufen
//
// Enthält: Kommandozeile lesen, Matrizen befüllen, Zeitmessung,
// Ergebnisprüfung und eine einheitliche CSV-Ausgabe, damit die
// Ergebnisse aller Stufen direkt vergleichbar sind.
```

```cpp
#pragma once

#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <random>
#include <string>
#include <vector>

namespace bench {

// Einstellungen aus der Kommandozeile:  ./stufeX <n> [wiederholungen]
struct Config {
  std::size_t n = 1024;   // Matrixgröße n x n
  int reps = 5;           // Messwiederholungen (nach dem Aufwärmlauf)
};

inline Config parse_args(int argc, char** argv) {
  Config c;
  if (argc > 1) c.n = std::strtoul(argv[1], nullptr, 10);
  if (argc > 2) c.reps = std::atoi(argv[2]);
  if (c.n == 0 || c.reps < 1) {
    std::fprintf(stderr, "Aufruf: %s <n> [wiederholungen]\n", argv[0]);
    std::exit(1);
  }
  return c;
}

// Matrizen mit reproduzierbaren Zufallswerten im Bereich [-1, 1] füllen.
inline void fill_random(float* m, std::size_t count, unsigned seed) {
  std::mt19937 gen(seed);
  std::uniform_real_distribution<float> dist(-1.0f, 1.0f);
  for (std::size_t i = 0; i < count; ++i) m[i] = dist(gen);
}

// Einfache Stoppuhr in Millisekunden.
class Timer {
 public:
  Timer() : start_(clock::now()) {}
  double ms() const {
    return std::chrono::duration<double, std::milli>(clock::now() - start_).count();
  }
 private:
  using clock = std::chrono::steady_clock;
  clock::time_point start_;
};

inline double median(std::vector<double> v) {
  std::sort(v.begin(), v.end());
  const std::size_t m = v.size() / 2;
  return v.size() % 2 ? v[m] : 0.5 * (v[m - 1] + v[m]);
}

// Ergebnisprüfung: 64 zufällige Einträge von C werden in double-Genauigkeit
// nachgerechnet. Das kostet nur O(64 * n) und funktioniert auch für große n.
inline bool verify(const float* A, const float* B, const float* C, std::size_t n) {
  std::mt19937 gen(42);
```

```
65    std::uniform_int_distribution<std::size_t> idx(0, n - 1);
66    for (int s = 0; s < 64; ++s) {
67      const std::size_t i = idx(gen), j = idx(gen);
68      double ref = 0.0;
69      for (std::size_t k = 0; k < n; ++k) ref += double(A[i * n + k]) * double(B[k * n + j]);
70      const double got = C[i * n + j];
71      const double tol = 1e-3 * std::max(1.0, std::sqrt(double(n)));
72      if (std::fabs(got - ref) > tol) {
73        std::fprintf(stderr, "FEHLER bei C[%zu][%zu]: %f statt %f\n", i, j, got, ref);
74        return false;
75      }
76    }
77    return true;
78  }
79
80  // Einheitliche Ergebniszeile (CSV), damit sich alle Stufen in eine Tabelle
81  // schreiben lassen:
82  //   stufe;geraet;n;erster_lauf_ms;median_ms;transfer_ms;gflops;korrekt
83  inline void report(const std::string& stage, const std::string& device, std::size_t n,
84                     double first_ms, double median_ms, double transfer_ms, bool ok) {
85    const double gflops = 2.0 * double(n) * double(n) * double(n) / (median_ms * 1e6);
86    std::printf("%s;%s;%zu;%.3f;%.3f;%.3f;%.2f;%s\n", stage.c_str(), device.c_str(), n,
87                first_ms, median_ms, transfer_ms, gflops, ok ? "ja" : "NEIN");
88  }
89
90  }  // namespace bench
```